

OS 21-22 Answers:

Question 1

(a) Define Operating System. Explain computer system structure.

An operating system is system software that acts as an interface between the user and computer hardware and manages system resources such as CPU, memory, and input/output devices.

A computer system consists of the following components

Hardware: Includes CPU, main memory, and input/output devices which provide basic computing resources.

Operating System: Controls and coordinates the use of hardware among various application programs and users.

System Programs: Provide a convenient environment for program development and execution, such as compilers, assemblers, and shells

Application Programs: Programs such as word processors, browsers, and media players that solve user problems.

The operating system acts as an intermediary between hardware and users to ensure efficient and safe use of system resources.

(b) Why do some systems store the operating system in firmware while others store it on disk?

Some systems store the operating system in firmware because firmware is non-volatile, reliable, and allows fast system startup. This is suitable for embedded systems where the operating system is small and rarely changed.

Other systems store the operating system on disk because modern operating systems are large and frequently updated. Storing the operating system on disk provides flexibility, easier updates, and supports complex functionalities required by general-purpose computers.

(c) What is an interrupt? How can we handle interrupt?

An interrupt is a signal generated by hardware or software that requests the immediate attention of the CPU. When an interrupt occurs, the CPU temporarily suspends the execution of the current program and transfers control to an interrupt service routine (ISR).

Interrupt handling process:

The interrupt signal is generated by a device or event.

The CPU saves the current execution state, including the program counter and registers.

Control is transferred to the appropriate interrupt service routine using the interrupt vector table.

The interrupt service routine executes and handles the interrupt.

After completion, the CPU restores the saved state and resumes the interrupted program.

(d) Differentiate between client-server computing and peer-to-peer computing.

Client–Server Computing	Peer-to-Peer Computing
Centralized architecture	Decentralized architecture
Server provides services to clients	Each node can act as both client and server
Easier management and security	Difficult to manage and secure
Server failure affects the system	No single point of failure

Question 2

(a) Explain the services provided by an operating system.

An operating system provides the following services:

Program Execution: Loads programs into memory, executes them, and terminates them properly.

Input/Output Operations: Provides a uniform interface for input and output operations and manages I/O devices.

File System Manipulation: Manages files and directories, including creation, deletion, reading, writing, and protection of files.

Communication: Provides mechanisms for inter-process communication such as message passing and shared memory.

Error Detection: Detects and handles errors in CPU, memory, and I/O devices to ensure reliable system operation.

Resource Allocation: Allocates system resources such as CPU time, memory, and I/O devices among competing processes.

Protection and Security: Ensures controlled access to system resources and protects data from unauthorized access.

(b) What are the two essential parts of a process? How is a process different from a program?

The two essential parts of a process are:

Program Code: The executable instructions of the program stored in memory.

Process Control Block (PCB): A data structure maintained by the operating system that contains information such as process ID, program counter, CPU registers, process state, memory management information, and I/O status.

Difference between process and program:

Program	Process
Passive entity stored on disk	Active entity in execution
No execution state	Has execution state and resources
No PCB associated	PCB associated with each process

(c) Describe Process Control Block (PCB) with proper figure.

A Process Control Block (PCB) is a data structure maintained by the operating system for each process. It stores all the information required to manage and control the execution of a process and enables context switching.

Contents of PCB include:

Process ID

Process state

Program counter

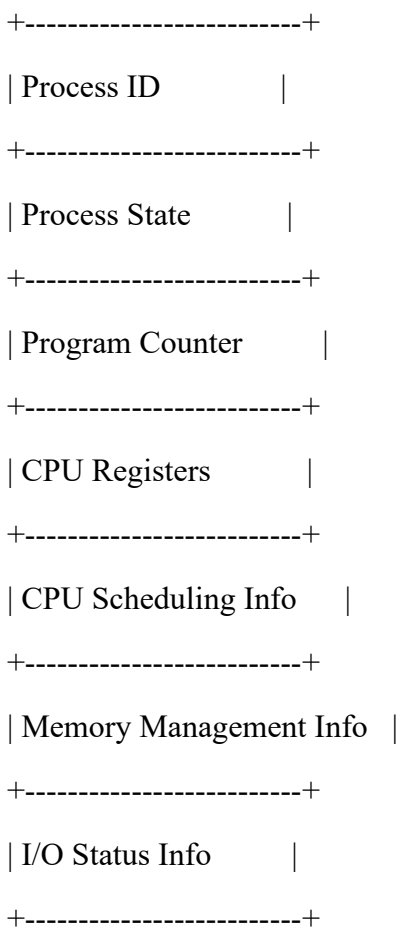
CPU registers

CPU scheduling information

Memory management information

I/O status information

Figure: Process Control Block



(d) Direct memory access is used for high-speed I/O devices to avoid increasing the CPU's execution load.

(i) CPU–device coordination:

The CPU initializes the DMA controller by providing the starting memory address, direction of transfer, and the number of bytes to be transferred. The DMA controller then transfers data directly between the I/O device and main memory. Upon completion, the DMA controller generates an interrupt to notify the CPU.

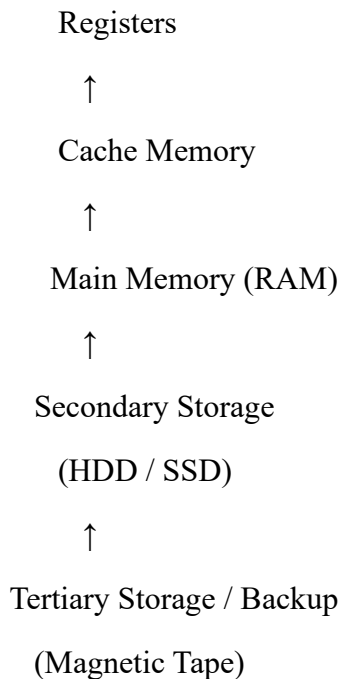
(ii) Interference with user programs:

Yes, limited interference may occur. The DMA controller may temporarily take control of the system bus, causing brief delays in CPU memory access (bus contention). Cache coherency issues may also arise if the CPU cache contains outdated data. However, these effects are minimal compared to the performance benefits of DMA.

Question 3

(a) Draw storage device hierarchy.

The storage device hierarchy represents the organization of storage systems based on speed, cost, and capacity.



Storage devices closer to the CPU are faster, smaller, and more expensive, while those farther away are slower, larger, and cheaper.

(b) What do you know about the critical section problem? How can we solve it?

The critical section is a part of a program where shared data or resources are accessed. The critical section problem is to design a mechanism that allows processes to access shared data without causing data inconsistency.

A correct solution must satisfy the following conditions:

Mutual Exclusion: Only one process can execute in the critical section at a time.

Progress: If no process is in the critical section, a waiting process must be allowed to enter.

Bounded Waiting: There must be a limit on how long a process waits to enter the critical section.

The critical section problem can be solved using mutex locks, semaphores, monitors, and hardware-based solutions such as test-and-set.

(c) Depict the control flow of system calls. Do you agree/disagree that system calls are time consuming? Give your statement.

When a user program requires a service from the operating system, it invokes a system call. The system call causes a trap that transfers control from user mode to kernel mode. The operating system identifies the requested service and executes the corresponding kernel routine. After completion, control returns to the user program and execution continues in user mode.

User Program



System Call Instruction



Kernel Mode



Operating System Service



Return to User Mode



User Program Continues

System calls are relatively time consuming compared to normal function calls because they involve mode switching, context saving and restoring, and security checks. However, this overhead is necessary to ensure system protection and controlled access to hardware resources.

(d) Mention some advantages of multithreaded programming.

Multithreaded programming offers several advantages:

Improved responsiveness of applications.

Better resource utilization due to shared address space.

Increased throughput through concurrent execution.

Efficient use of multi-core processors.

Question 4

(a) Describe the necessary and sufficient conditions for deadlock to occur.

Deadlock is a situation in which a set of processes are permanently blocked because each process is holding a resource and waiting for another resource held by another process.

The necessary and sufficient conditions for deadlock are:

Mutual Exclusion: At least one resource must be held in a non-shareable mode.

Hold and Wait: A process holding at least one resource is waiting to acquire additional resources held by other processes.

No Preemption: Resources cannot be forcibly taken from a process and must be released voluntarily.

Circular Wait: A circular chain of processes exists in which each process is waiting for a resource held by the next process.

Deadlock occurs if and only if all these conditions hold simultaneously.

(b) Consider the following set of processes with CPU burst time and compare FCFS, SJF (Non-Preemptive), and Round Robin (Time Quantum = 10) scheduling algorithms to find the minimum average waiting time.

Process	Burst Time
P1	10
P2	29
P3	3
P4	7
P5	12

FCFS Scheduling

Order: P1 → P2 → P3 → P4 → P5

Waiting times:

P1 = 0, P2 = 10, P3 = 39, P4 = 42, P5 = 49

Average waiting time:

$(0+10+39+42+49)/5=28$

SJF Scheduling (Non-Preemptive)

Order: P3 → P4 → P1 → P5 → P2

Waiting times:

P3 = 0, P4 = 3, P1 = 10, P5 = 20, P2 = 32

Average waiting time:

$(0+3+10+20+32)/5=13$

Round Robin Scheduling (Time Quantum = 10)

Gantt chart:

P1 P2 P3 P4 P5 P2 P5 P2
0 10 20 23 30 40 50 52 61

Completion times:

P1 = 10, P2 = 61, P3 = 23, P4 = 30, P5 = 52

Waiting times (WT = CT – BT):

P1 = 0, P2 = 32, P3 = 20, P4 = 23, P5 = 40

Average waiting time:

$0+32+20+23+40/5=23$

Conclusion

Algorithm	Average Waiting Time
FCFS	28
SJF (Non-Preemptive)	13
Round Robin	23

SJF (Non-Preemptive) gives the minimum average waiting time.

(c) Shortly explain the scheduling criteria for CPU scheduling algorithms.

The scheduling criteria for CPU scheduling algorithms are:

CPU Utilization: Percentage of time the CPU remains busy; should be maximized.

Throughput: Number of processes completed per unit time; higher is better.

Turnaround Time: Time from submission of a process to its completion; should be minimized.

Waiting Time: Total time a process spends waiting in the ready queue; should be minimized.

Response Time: Time from submission of a request to the first response; important for interactive systems.

Question 5

(a) Consider a computer system that runs 5,000 jobs per month with no deadlock prevention or avoidance. Discuss the arguments for and against installing a deadlock-avoidance algorithm.

(i) Arguments for installing deadlock-avoidance algorithm:

Installing a deadlock-avoidance algorithm prevents deadlocks and eliminates the need to abort and rerun jobs. This improves system reliability and avoids wastage of CPU time and operator effort. Since the system has 30 percent idle time, the additional 10 percent execution overhead can be accommodated without reducing throughput. Deadlock avoidance also improves system predictability and user satisfaction.

(ii) Arguments against installing deadlock-avoidance algorithm:

The deadlock-avoidance algorithm increases average execution time per job and increases turnaround time by about 20 percent. Deadlocks occur infrequently and affect only a small number of jobs, resulting in relatively low cost. Therefore, the added complexity and performance degradation may not be justified compared to the limited losses caused by occasional deadlocks.

(b) How can we recover from deadlock? Shortly describe.

Deadlock recovery techniques are applied after a deadlock has been detected. The main recovery methods are:

Process Termination:

One or more processes involved in the deadlock are terminated to break the deadlock. Processes may be aborted all at once or one at a time until the deadlock is resolved.

Resource Preemption:

Resources are forcibly taken from some processes and allocated to others. The system may roll back processes to a safe state to ensure correct execution.

(c) Consider $\alpha = 80\%$, $\varepsilon = 20$ ns for TLB search, and 100 ns for memory access. Find the effective access time.

On a TLB hit, the access time is:

$$20+100=120 \text{ ns}$$

On a TLB miss, the access time is:

$$20+100+100=220 \text{ ns}$$

The effective access time (EAT) is:

$$\text{EAT} = 0.8 \times 120 + 0.2 \times 220 = 96 + 44 = 140 \text{ ns}$$

Therefore, the effective access time is 140 ns.

(d) What do you know about resource allocation graph? Consider the given graph and determine whether deadlock exists. Justify your answer.

A Resource Allocation Graph (RAG) is a directed graph that represents resource allocation and resource requests by processes. Processes are represented by circles and resources by rectangles. A request edge indicates that a process is waiting for a resource, while an assignment edge indicates that a resource has been allocated.

In the given graph, processes P1, P2, and P3 form a circular wait. Resource R1 and R3 have single instances, while resource R2 has multiple instances, all of which are allocated. Each process is waiting for a resource held by another process, and no process can proceed to release its resources.

Therefore, the system is in a deadlock state.

Question 6

(a) Consider the following snapshot of a system.

(i) What is the content of the matrix Need

(ii) If a request from process P1 arrives for (1,4,6,0), can the request be granted immediately?

The Need matrix is calculated as:

$$\text{Need} = \text{Max} - \text{Allocation}$$

Process	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

For process P1,

Need(P1)=(0,7,5,0)

The request from P1 is (1,4,6,0), which exceeds its need for resources A and C.

Therefore, the request cannot be granted immediately.

(b) Define paging. Shortly describe address translation scheme with suitable example.

Paging is a memory management technique in which logical memory is divided into fixed-size blocks called pages and physical memory is divided into fixed-size blocks called frames. The operating system maintains a page table to map pages to frames, allowing non-contiguous memory allocation.

In paging, a logical address is divided into a page number and an offset. The page number is used to index the page table to obtain the corresponding frame number. The physical address is formed by combining the frame number with the offset.

Example:

If page size is 1 KB and the logical address is 2050, then page number = 2 and offset = 2. If page 2 is mapped to frame 5, the physical address is $(5 \times 1024) + 2 = 5122$.

(c) Design a multithreaded program to count and print the total number of words in two files. Give two use-case scenarios.

A multithreaded program can be designed by creating two threads, where each thread reads one file and counts the number of words independently. Each thread updates a shared variable containing the total word count using proper synchronization. The main thread waits for both threads to complete execution and then prints the final total word count.

Use-case scenarios:

Parallel analysis of large log files to reduce processing time.

Document processing and indexing systems where word counts are needed from multiple files simultaneously.

Question 7

(a) Compare between physical address space and logical address space.

Logical Address Space	Physical Address Space
Generated by the CPU	Generated by memory hardware
Also called virtual address	Actual address in main memory
Used by programs	Used by physical memory
Needs address translation	No translation required
Independent of memory size	Limited by physical memory size

(b) What is fragmentation? Consider the following segment table. What are the physical addresses for the given logical addresses?

Fragmentation refers to inefficient utilization of memory space. Internal fragmentation occurs when unused memory exists inside allocated blocks, while external fragmentation occurs when free memory exists but is not contiguous.

In segmentation, a logical address is of the form (segment number, offset). The physical address is obtained by adding the offset to the base address of the segment, provided the offset is less than the segment length.

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	5806

Physical addresses:

(0, 430): $219 + 430 = 649$

(1, 10): $2300 + 10 = 2310$

(2, 500): Offset exceeds segment length → Invalid address (trap)

(3, 400): $1327 + 400 = 1727$

(c) Define address binding. Explain multistep processing of a user program highlighting the different stages of address binding.

Address binding is the process of mapping logical addresses of a program to physical addresses in main memory.

The stages of address binding are:

Compile-Time Binding: Absolute addresses are generated by the compiler when the memory location is known at compile time.

Load-Time Binding: Relocatable code is generated, and address binding is performed by the loader when the program is loaded into memory.

Execution-Time Binding: Address binding is performed during program execution using hardware support such as the Memory Management Unit (MMU).

Multistep processing of a user program:

Source Program → Compiler → Object Program → Linker → Executable Program → Loader → Main Memory

Question 8

(a) Consider the following reference string. Calculate the number of page faults using FIFO page replacement algorithm.

Reference string:

1, 2, 3, 2, 5, 6, 3, 4, 6, 3, 7, 1, 5, 3, 6, 3, 4, 2, 4, 5, 4, 5

Number of frames = 3

Using the FIFO page replacement algorithm, pages are replaced in the order in which they entered memory. After simulating the reference string with three frames, the total number of page faults is 13.

(b) What is Belady's Anomaly? Describe with example.

Belady's Anomaly is a phenomenon in which increasing the number of page frames results in an increase in the number of page faults when using certain page replacement algorithms.

This anomaly occurs in the FIFO page replacement algorithm because it does not satisfy the stack property.

Example:

For the reference string

1 2 3 4 1 2 5 1 2 3 4 5,

FIFO produces 9 page faults with 3 frames and 10 page faults with 4 frames.

Thus, increasing the number of frames increases the number of page faults, which is known as Belady's Anomaly.

(c) Explain demand paging with proper figure.

Demand paging is a virtual memory management technique in which pages are loaded into main memory only when they are required during execution. Pages that are not immediately needed remain on secondary storage.

When a process accesses a page that is not present in memory, a page fault occurs. The operating system fetches the required page from disk, loads it into a free frame in memory (or replaces an existing page), updates the page table, and resumes execution.

Figure: Demand Paging Mechanism

CPU → Page Table → Main Memory

↓

Page Fault

↓

Disk

Demand paging improves memory utilization, reduces I/O operations, and allows execution of large programs using limited main memory.